Алгоритмы и структуры данных

1 семестр (ITMO IS)

Оценка сложности по времени

$O(f(n))$: существуют $c > 0$, n_0 такие, что $T(n) \leq c \cdot f(n)$ при всех $n \geq n_0$. $\Omega(f(n))$: $T(n) \geq c \cdot f(n)$. $\Theta(f(n))$: одновременно O и Ω.

Правила: $O(f + g) = O(\max(f, g))$; $O(c \cdot f) = O(f)$.

Иерархия: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Бинарный поиск: $O(\log n)$. Сортировка пузырьком: $O(n^2)$.

Оценка сложности по памяти

Вспомогательная — доп. память сверх входных данных. **Общая** — включает вход.

$O(1)$: итеративный алгоритм. $O(\log n)$: рекурсия с log-глубиной. $O(n)$: доп. массив. $O(n^2)$: матрица смежности.

Сортировка вставками

Для каждого элемента ищем его место среди уже отсортированных.

```
insertionSort(A, n):
  for i = 1 to n-1:
    key = A[i]; j = i-1
    while j >= 0 and A[j] > key:
      A[j+1] = A[j]; j--
    A[j+1] = key
```

Время: $O(n^2)$ worst/avg, $O(n)$ best. Память: $O(1)$. Устойчива.

Сортировка слиянием

Рекурсивно делим массив пополам, сортируем, сливаем за $O(n)$.

```
mergeSort(A, lo, hi):
  if lo >= hi: return
  mid = (lo+hi)/2
  mergeSort(A, lo, mid)
  mergeSort(A, mid+1, hi)
  merge(A, lo, mid, hi)

merge(A, lo, mid, hi):
  tmp = A[lo..hi]
  li=lo, ri=mid+1, k=lo
  while li<=mid and ri<=hi:
    if tmp[li]<=tmp[ri]: A[k++]=tmp[li++]
    else: A[k++]=tmp[ri++]
  copy remaining
```

Время: $O(n \log n)$. Память: $O(n)$. Устойчива.

Быстрая сортировка

Выбираем опорный элемент, разделяем массив, рекурсируем.

```
quickSort(A, lo, hi):
  if lo >= hi: return
  p = partition(A, lo, hi)
  quickSort(A, lo, p-1)
  quickSort(A, p+1, hi)
```

Время: $O(n \log n)$ avg, $O(n^2)$ worst. Память: $O(\log n)$ стек. Нет гарантий без рандомизации.

```
partition(A, lo, hi):  // Lomuto
  pivot=A[hi]; i=lo-1
  for j=lo to hi-1:
    if A[j]<=pivot: swap A[++i], A[j]
  swap A[i+1], A[hi]
  return i+1
```

Сортировка подсчётом

Для целых чисел в диапазоне $[0, k]$: считаем частоты, строим prefix sum.

```
countingSort(A, n, k):
  count[0..k] = {0}
  for i=0..n-1: count[A[i]]++
  for i=1..k: count[i]+=count[i-1]
  for i=n-1 downto 0:
    out[--count[A[i]]] = A[i]
```

Время: $O(n + k)$. Память: $O(k)$. Устойчива. Только для целых ≥ 0.

Цифровая сортировка (Radix LSD)

Сортируем поразрядно от младшего разряда к старшему устойчивой сортировкой.

```
radixSort(A, n, d):
  for digit = 0 to d-1:
    stableSort(A, n, key=(x//base^digit)%base)
```

Время: $O(d \cdot (n + k))$. Память: $O(n + k)$. Устойчива. d — число разрядов.

Стек

LIFO. Реализация на массиве с указателем `top`.

```
push(x):    stack[++top] = x
pop():      return stack[top--]
peek():     return stack[top]
is_empty(): return top == -1
```

Время: $O(1)$. Память: $O(n)$. Применение: DFS, скобочные выражения, undo.

Очередь

FIFO. Кольцевой буфер для эффективной реализации.

```
enqueue(x): q[back] = x
            back=(back+1)%N
dequeue():  x=q[front]
            front=(front+1)%N
            return x
```

Время: $O(1)$. Память: $O(n)$. Применение: BFS, планирование задач.

Односвязный список

Каждый узел хранит значение и указатель на следующий.

```
insert_head(val):
  node=new Node(val)
  node.next=head; head=node

delete(val):
  prev.next = cur.next
```

insert_head: $O(1)$. insert_tail / search / delete: $O(n)$. Нет произвольного доступа.

```
search(val):
  traverse from head
```

Двусвязный список

Каждый узел: `val`, `next`, `prev`. Позволяет вставку и удаление за $O(1)$ при наличии указателя.

```
insert_after(node, val):
  new_node = Node(val)
  new_node.next = node.next
  new_node.prev = node
  if node.next: node.next.prev = new_node
  node.next = new_node

delete(node):
  node.prev.next = node.next
  node.next.prev = node.prev
```

Все операции (при известном узле): $O(1)$. Памяти больше (2 указателя).

Циклический список

Последний узел указывает на первый. Хранить указатель на `tail`.

Обход: `while cur.next != head`. Удобен для кругового планирования и деков.

Стек на связном списке

Динамический размер, нет ограничений на ёмкость.

```
push(x):
  node=new Node(x)
  node.next=top; top=node

pop():
  val=top.val
  top=top.next; return val
```

Время: $O(1)$. Нет ограничений массива. Накладные расходы: указатели.

Очередь на связном списке

Указатели `front` и `back` для $O(1)$ enqueue/dequeue.

```
enqueue(x):
  node=new Node(x)
  if empty: front=back=node
  else: back.next=node; back=node

dequeue():
  val=front.val
  front=front.next; return val
```

Время: $O(1)$. Динамический размер.

Пирамида — базовые операции

Max-heap: `heap[parent] >= heap[child]`. Индексы: `parent(i)=(i-1)/2`, `left=2i+1`, `right=2i+2`.

```
sift_up(i):
  while i>0 and heap[parent(i)] < heap[i]:
    swap heap[i], heap[parent(i)]
    i = parent(i)

sift_down(i, n):
```

insert: $O(\log n)$. peek_max: $O(1)$.

```
  while 2i+1 < n:
    largest=i
    if heap[2i+1]>heap[largest]: largest=2i+1
    if 2i+2<n and heap[2i+2]>heap[largest]:
      largest=2i+2
    if largest==i: break
    swap heap[i], heap[largest]; i=largest

insert(x): heap[n++]=x; sift_up(n-1)
peek_max(): return heap[0]
```

Построение пирамиды за O(n)

Запускаем `sift_down` от последнего нелистового узла вниз.

```
buildHeap(A, n):
  for i = n/2-1 downto 0:
    sift_down(A, i, n)
```

Время: $O(n)$. Сумма работы: $\sum_{d=0}^{\log n}(h-d)\cdot 2^d = O(n)$. Листья ($\approx n/2$) не требуют sift_down.

Пирамидальная сортировка

Строим кучу, затем поочерёдно извлекаем максимум.

```
heapSort(A, n):
  buildHeap(A, n)
  for i = n-1 downto 1:
    swap A[0], A[i]
    sift_down(A, 0, i)
```

Время: $O(n \log n)$. Память: $O(1)$. Нестабильна. Не cache-friendly.

Удаление из пирамиды

```
extractMax():
  max=heap[0]
  heap[0]=heap[--n]
  sift_down(0, n)
  return max

deleteAt(i):
  heap[i]=heap[--n]
  sift_up(i)
  sift_down(i, n)
```

Оба: $O(\log n)$. `deleteAt`: один из sift сработает (вверх или вниз).

Очередь с приоритетом

Реализация на max-heap. Применение: алгоритм Дейкстры, алгоритм Прима.

```
insert(x, priority):
  heap.push((priority, x))
  sift_up(last)

extractMax():
  вернуть heap[0], sift_down

increaseKey(i, k):
  heap[i].priority=k; sift_up(i)
```

Все операции: $O(\log n)$.

```
decreaseKey(i, k):
  heap[i].priority=k; sift_down(i,n)
```

ДП: базовые задачи

Кузнечик (1 или 2 шага, минимальная стоимость):

```
dp[0]=0
for i=1 to n:
  dp[i] = cost[i] + min(dp[i-1],
          dp[i-2] if i>=2 else INF)
```

Время: $O(n)$. Память: $O(n)$ или $O(1)$ с перекатыванием.

Рюкзак 0/1 (n предметов, ёмкость W):

```
dp[0..W] = {0}
for i=1 to n:
  for w=W downto items[i].weight:
    dp[w] = max(dp[w],
      dp[w-items[i].weight]+items[i].value)
```

Время: $O(nW)$. Память: $O(W)$.

НВП (наибольшая возрастающая подпоследовательность):

```
for i=0 to n-1:
  dp[i]=1
  for j=0 to i-1:
    if A[j]<A[i]: dp[i]=max(dp[i], dp[j]+1)
ans = max(dp)
```

Время: $O(n^2)$. Оптимально: $O(n \log n)$ с терпением + бинарный поиск.

Быстрый выбор (Quickselect)

Находит k-й по возрастанию элемент за $O(n)$ в среднем.

```
quickselect(A, lo, hi, k):
  if lo==hi: return A[lo]
  p = partition(A, lo, hi)  // Lomuto
  if k==p: return A[p]
  elif k<p: return quickselect(A, lo, p-1, k)
  else: return quickselect(A, p+1, hi, k)
```

Время: $O(n)$ avg, $O(n^2)$ worst. Память: $O(\log n)$ стек.

Мастер-теорема

$T(n) = a \cdot T(n/b) + f(n)$, где $a \geq 1, b > 1$.

Случай 1: $f(n) = O\left(n^{\log_b(a)-\varepsilon}\right) \Rightarrow T(n) = \Theta\left(n^{\log_b(a)}\right)$

Случай 2: $f(n) = \Theta\left(n^{\log_b(a)}\right) \Rightarrow T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$

Случай 3: $f(n) = \Omega\left(n^{\log_b(a)+\varepsilon}\right)$ и регулярность $\Rightarrow T(n) = \Theta(f(n))$

Пример: сортировка слиянием $T(n) = 2T(n/2) + O(n) \Rightarrow$ случай 2 $\Rightarrow O(n \log n)$.

Дек и очередь с минимумом

Дек (двусторонняя очередь): вставка/удаление с обоих концов за $O(1)$.

Очередь с минимумом — монотонная дека:

```
enqueue_min(x):
  while not dmin.empty() and
        dmin.back() > x:
    dmin.pop_back()
  dmin.push_back(x)
  q.push_back(x)

dequeue_min():
  val=q.pop_front()
  if dmin.front()==val: dmin.pop_front()
  return val

get_min(): return dmin.front()
```

Все операции: $O(1)$ амортизированно. Монотонная дека хранит только потенциальные минимумы.

Бинарный поиск

lower_bound — первый элемент $\geq$ target:

```
lower_bound(a, target):
  lo=0, hi=n
  while lo < hi:
    mid = (lo+hi)/2
    if a[mid] < target: lo=mid+1
    else: hi=mid
  return lo

// Поиск по ответу:
lo=min_ans, hi=max_ans
while lo < hi:
  mid=(lo+hi)/2
  if check(mid): hi=mid
  else: lo=mid+1
return lo

// Вещественный:
while hi-lo > eps:
  mid=(lo+hi)/2
  if f(mid)>=0: hi=mid else lo=mid
```

Время: $O(\log n)$. Память: $O(1)$.

ДП: продвинутые техники

Редакционное расстояние (Левенштейн):

```
dp[0][j]=j, dp[i][0]=i
for i=1..n, j=1..m:
  if s[i-1]==t[j-1]:
    dp[i][j]=dp[i-1][j-1]
  else:
    dp[i][j]=1+min(dp[i-1][j],
                   dp[i][j-1],
                   dp[i-1][j-1])
```

Время: $O(nm)$. Память: $O(nm)$ или $O(m)$ с перекатыванием.

ДП по подмножествам (битовые маски) — кратчайший гамильтонов путь (TSP):

```
// dp[mask][v] = мин. стоимость пути,
// посетившего все города в mask,
// заканчивающегося в v
dp[1<<s][s] = 0
```

Время: $O(2^n \cdot n^2)$. Память: $O(2^n \cdot n)$.

```
for mask=1..(1<<n)-1:
  for v in mask:
    for u not in mask:
      dp[mask|(1<<u)][u] = min(
        dp[mask|(1<<u)][u],
        dp[mask][v]+dist[v][u])
```